

SmoothMQTT v1.3

Why bother coding, when connecting hardware can be click and play? SmoothMQTT will give you the power to connect external hardware like LEDs, sensors, motors, etc. with your Unity app or game, without the entry-barrier of programming. All components provided in this package will allow you to easily access Unity functions in response to messages from other apps or hardware appliances.

[Buy SmoothMQTT](#) on the Unity3D AssetStore

MQTT is a message protocol that allows you to publish values to a topic that any client can subscribe and react to. Imagine a sensor in your living room publishes its current temperature measurement to MQTT and your game changes the light intensity or color in your scene.

This asset provides you with components which make it easy to run and/or use MQTT infrastructure within your game or app. All components included are designed to resemble your regular workflow, no matter if it's inspector-based or code-based development. Subscribing to a topic is as easy as

1. adding a component
2. configuring the topic
3. selecting an action to be triggered

Just like you are used to from UI buttons and EventTriggers in Unity.

What is MQTT?

MQTT is a message broker system to distribute messages from any type of client (sensors, computers, etc.) over ethernet. These messages are published on a specified topic to the server(broker). Clients can subscribe to one or multiple topics to be informed whenever a message to this topic has been published. Besides the topic, a message can also carry a payload to transfer data to all subscribers.

External broker

While SmoothMQTT brings its own broker, sometimes it is desirable to have an external program take over this role.

Mosquitto

One of the most popular programs is called [Mosquitto](#). It can be installed as a service on any major OS and provide an MQTT broker.

Cloud Broker

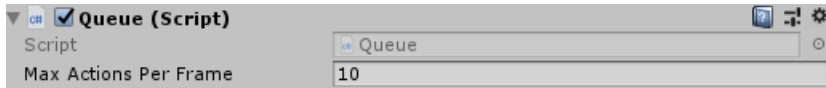
We regularly test SmoothMQTT against AWS and are optimistic that other mqtt cloud broker providers will work as well.

Quickstart

- ☒ Open the quickstart guide
- ☐ Open Unity 2018.4 or later
- ☐ Import the SmoothMQTT.unitypackage (or install from AssetStore)
- ☐ Add the necessary MQTTManager by clicking GameObject->SmoothMQTT->MQTTManagerWithBroker
- ☐ To send MQTT messages from an object interaction, select the object and click Tools->SmoothMQTT->Send->React to UnityEvents from the main menu
- ☐ Configure the topic to publish to in the component SendOnEvent
- ☐ To react to MQTT messages, select an object and add a subscriber with Tools->SmoothMQTT->Receive->StringPayload.
- ☐ Configure the topic to react to in the component Subscriber and add Actions (which accept no parameters or String) to invoke when an MQTT message is received.
- ☐ Find more ways of receiving and sending messages in the Tools->SmoothMQTT menu

Core Components

Queue



At the very core of SmoothMQTT there is a queue taking care of all configured subscriptions. All messages that arrive as result of a subscription are collected here for execution at the next possible time. With `maxActionsPerFrame` you can configure how many events shall be processed per frame. Lower number means higher framerate but potential delay of reaction to MQTT messages. Higher number means all messages are dealt with right away, but it might impact framerate.

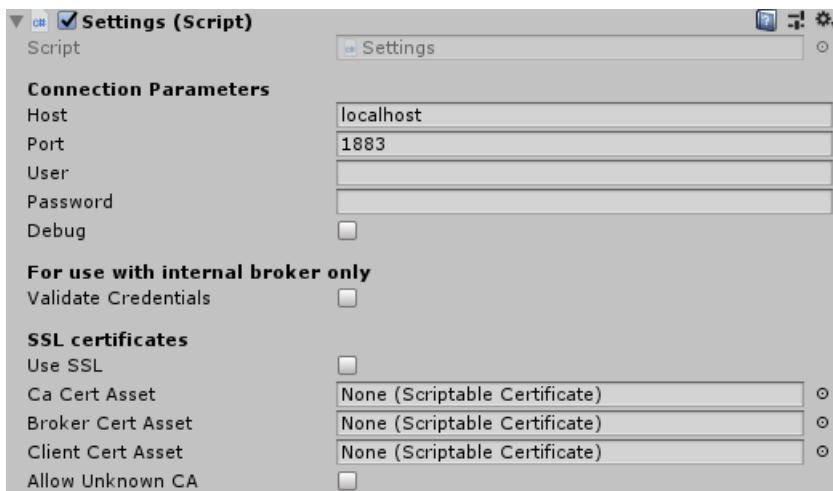
! TECHNICAL BACKGROUND

The queue is necessary, because MQTT subscriptions run asynchronously, but changes to the Unity Scene or Gameobjects have to be made on the main thread.

💡 ADVANCED USERS

A way of improving performance is to keep the subscribe `action` small in scope or ideally only use it to start a `Coroutine`.

Settings

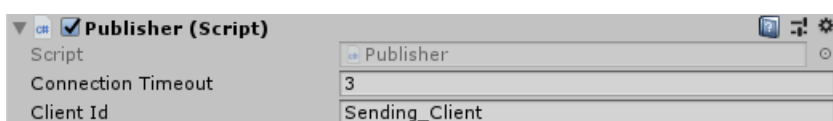


No matter whether you run the broker externally or right from Unity, the Settings component is where you configure the `host` (URL or IP address), `port` and if available `username` and `password` for login at the broker. The Settings component uses a Singleton pattern and thus is only allowed once in your scene. If you are using the internal broker (i.e. `MqttManagerWithBroker` prefab) you can also select whether the broker requires credentials.

Since in some projects it can be beneficial to not start the broker or subscribers right away when the scene starts, there is now a setting to `Connect On Start`. Similar `Auto Reconnect` deals with disconnects (for example due to flaky network connection)

For SSL options check [Security certificates -> Settings](#)

Publisher

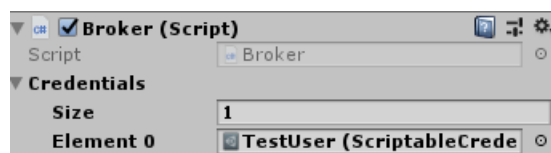


Publisher component allows you to publish MQTT messages for any topic with any payload. Thus it is not necessary to have more than one in the scene. The `MqttManager` Prefab has a Publisher component attached. However, if you need to have two or more Publisher components with different client IDs registered with your broker, you can just add them to your scene and name them accordingly.

! INFO

Client IDs may not be empty and may not be used twice on different components

Broker

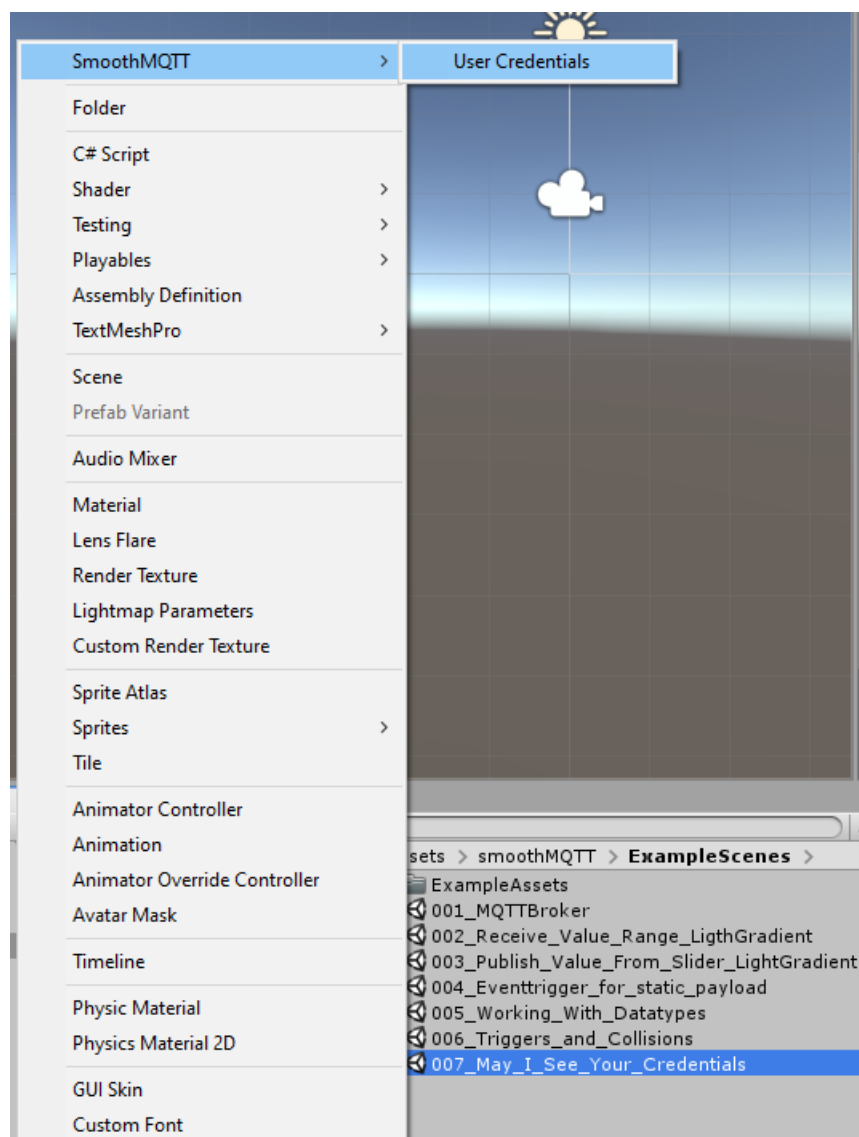


This component provides you with an MQTT broker from within Unity. There is nothing to configure really, as it takes the same settings as all clients in your app. If this component is present, Unity will take care of receiving and distributing MQTT messages without the need of additional software. The broker uses a Singleton pattern and thus is only allowed once in your scene. As of now, if you use your Unity game as MQTT broker, there is no way to setup user-specific permission sets. This might be added in the future and has low priority for now.

⚠ CAUTION

From Unity 2019 onwards the Broker does not fully work from within the editor (something to do with Networking rules). It will allow any MQTT activity on localhost through, but no connections from other devices on your network. It DOES, however, work just fine in builds. So for development the currently suggested workaround is to use an external broker like [mosquitto](#) during development and activate broker component just before deployment.

Credentials

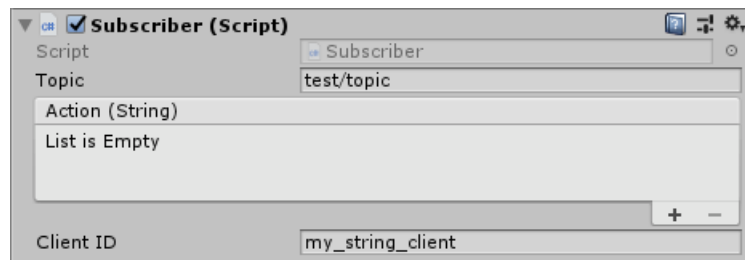


If you run a broker directly from Unity, you might want to be able to restrict access to users with a valid password. As of version 1.0.6 you can create credential assets by using the Create menu in your Project view (see screenshot). In the asset you can choose a username and password (**case-sensitive**). Finally you need to add all valid users to the credentials list of the broker component in your scene. Check example scene `007_May_I_See_Your_Credentials`.

⚠ CAUTION

As of the current version this just restricts access by suppressing anonymous connections, but it is **not secure** by any means. Passwords are sent in Plaintext (if SSL encryption is not used) and users cannot be restricted to read/write only certain topics (We have yet to find a good way to integrate this without violating our claim: No Code! Just Connect!, but we're working on it.)

Subscriber Component



Listening to MQTT messages is as easy as giving any Gameobject a Subscriber component. It can be configured with a specific topic to listen on (or use “#” to listen for any message). Any Method accepting a string (the payload) as parameter can subsequently be added to the Action list below. Think for example SetText(string) method of TMP_Text fields or your own methods.

! INFO

Client IDs may not be empty and may not be used twice on different components

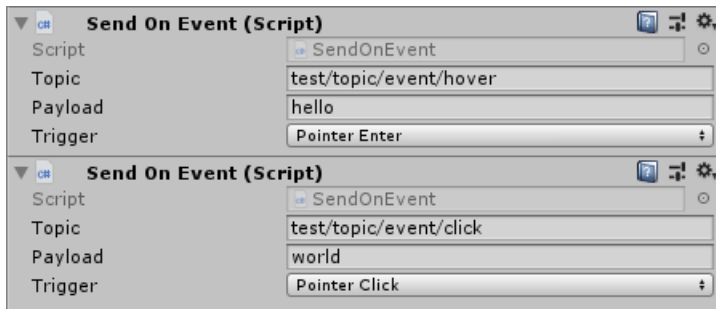
For other data types see [Subscriber Converter](#).

Sending / Publishing Messages

All messages are sent through the central Publisher component. Therefore, no client ID is required on any of the following components.

SendOnEvent

This component can be attached to an object that is supposed to send an MQTT message upon an Eventsystem event (like Pointer Enter, Begin Drag, etc.) It covers the same Events as Unity's EventTrigger component. You can define a topic and a payload on the component.



TIP

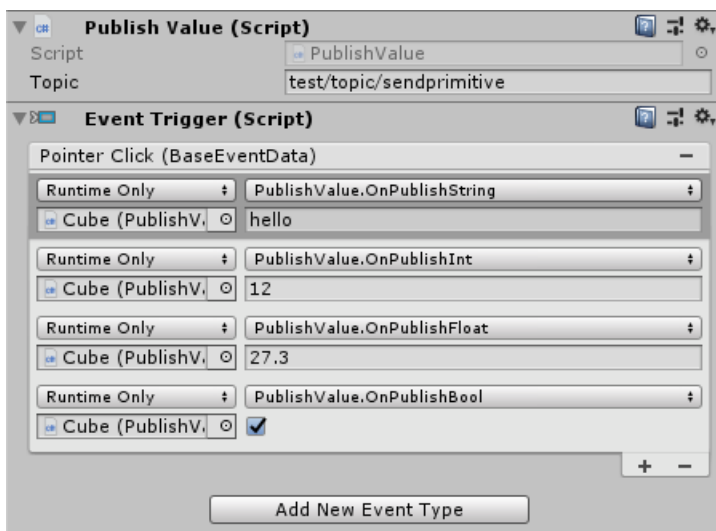
Just like with Unity Events, you need to setup a few things. There needs to be a collider on the object, an Eventsystem in the scene and a PhysicsRaycaster on the Camera.

PublishValue

PublishValue allows you to send data to the topic specified in this component. You can hook up the following methods to any UnityEvent (e.g. a slider's OnValueChanged event, a Button's OnClick, or an EventTrigger):

- OnPublishString
- OnPublishFloat
- OnPublishDouble
- OnPublishInt
- OnPublishBool The values passed are converted to string before publishing as MQTT message.

NOTE: Hooking up a slider will send an MQTT message everytime the slider changes (i.e. everytime the mouse moves while dragging). This creates a lot of messages and might affect performance if used too extensively.



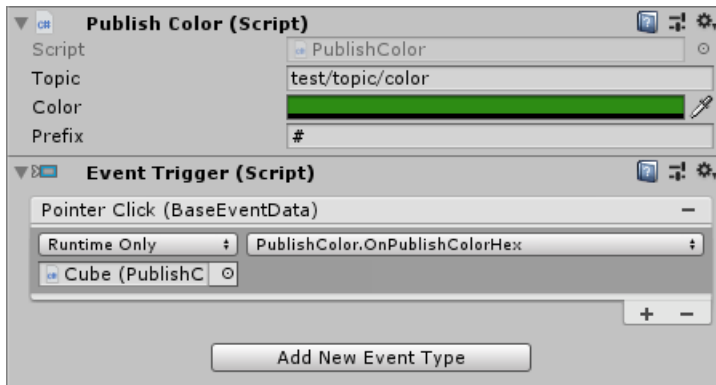
Other Publish Components

For more complex values, the following components are available

PublishColor

This component will allow you to select a color on a color picker to be sent via MQTT. Colors will be converted to hexRGB (e.g. 100% red => FF0000). The default prefix is # however it can be changed to anything that is not 0-9 or A-F.

message format `#RRGGBB` (R=Red, G=Green, B=Blue)



PublishQuaternion

If you want to send rotations, use this component. You can select whether to publish the rotation of

- a reference transform (global rotation)
- quaternion values x, y, z, w
- Euler angles x, y, z

Set the type accordingly (Reference Transform, Quaternion, Euler Angles). Note: Only the values for the selected type are considered. If for example you select Euler Angles, a linked Transform or values in the quaternion section in inspector will be ignored and the message will be build just from the vector3 (interpreted as Euler Angles). The message payload will always be a quaternion though.

message format `(x, y, z, w)`

PublishVector3

Vector3 MQTT messages can be used for positions, scale, and such. For the moment PublishVector3 will send the Vector3 configured in the inspector. See [roadmap](#) for future plans.

PublishFromTMPInputField

Sending data from an input field is a common use-case. In contrast to the other publish components, this one needs to be on the same gameobject as the according TMP_InputField. When triggered by button or eventtrigger (see picture below) it will send the text entered into the inputfield.

PublishTriggerCollision

In general, this component allows reporting of collisions or trigger events to MQTT when the gameobject enters or exits (see below for On...Stay event) another collider.

Prerequisites for trigger:

- Both interacting objects need to have colliders
- One of the colliders has to be a trigger

Prerequisites for collision:

- Both interacting objects need to have colliders
- In case of Mesh Colliders, it either needs to be convex or kinematic.

If it does not work for any reason, cross-check [Unity Manual: Collision Matrix](#) for detailed collision information

Inspector Config: *MQTT* As usual you can configure the topic on which a collision event will be published. Since it can be beneficial, I added some

dynamic values (see table and inspector examples below) to be automatically replaced in the topic string. If you want to use these placeholders, make sure `Format Topic String` is checked. Finally select with `Is Trigger`, whether you want to react to trigger events (checked) or to collision events (unchecked).

Source and Target The `Source Collider` is supposed to be the main collider involved in an event. Typically this is the first collider of a gameobject and it will be automatically set to that, but if for some reason you want to use a different one, feel free to override it. The target settings allow you to filter collisions to only report on specific objects (`Filter Target Game Object`) or Tags (`Filter Target Tag`) e.g. "Player". If you set both filters, they are BOTH required (logical AND). Filtering by tag usually does not make sense, if you link the exact gameobject, unless you want to change the `Filter Target GameObject` at runtime (through code) and constrain events to e.g. `Tag=Enemy`.

Actions to Publish Here you can select, which Events you want to publish as MQTT message.

OnStay events If you want to publish a message while an object is within a trigger volume, you can check `Emit MQTT On Stay`. It is crucial, that you set an emit interval that is as long as possible for your use case. Without an interval it would send a message every frame (i.e. >>30 messages per second) and clutter up your broker. Therefore an interval is mandatory and should not be lower than 0.5 seconds. The unit of `Emit Interval` is seconds (between messages). Be careful with these settings, as they can easily kill your performance and/or fill your message queue and cause delays in other MQTT messages.

NOTE: "Collision stay events are not sent for sleeping Rigidbodies." [Unity Manual](#)

placeholder overview

name placeholder	description	possible values (always lower case)
name %n	Shows the name of the gameobject having the publish component	actual name e.g. "cube"
type %t	Shows whether it's a collision or a trigger event	"collision" or "trigger"
action %a	Shows the kind of event	"enter", "stay", or "exit"

message format The default message format is `type "name" action "other.name"`, so a "Cube" object colliding with a "Plane" would send `Collision "Cube" Enter "Plane"` (Names are quoted in payload, because they can contain spaces)

The message format changes when using `Format Topic String` in a way that all information used in the topic string will be omitted from the payload.

Example 1: Upon colliding with "Plane" a "Cube" with format string `"%n/%t"` would send `Enter "Plane"` to topic `cube/collision`.

Example 2: Upon colliding with "Plane" a "Cube" with format string `"%n/%t/%a"` would just send `"Plane"` to topic `cube/collision/enter`.

Receiving and Reacting to Messages

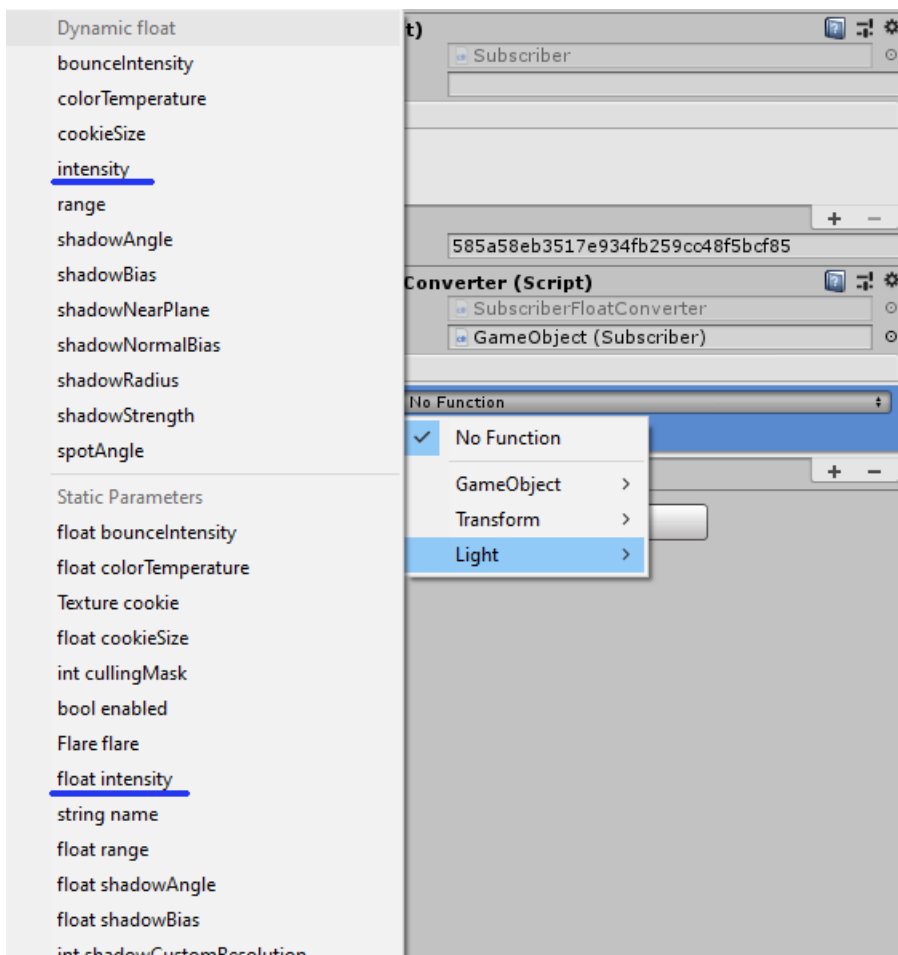
Converters provide a UnityAction acting on other datatypes than string. When added through the Tools menu, a regular [Subscriber](#) component will be attached to the selected gameobject along with a universal converter component configured for the selected datatype. Add desired actions in the converter component, rather than the subscriber.

⚠ CAUTION

The UniversalConverter component will replace all prior converters. Therefore these converter components are marked 'deprecated' already and will be removed in the next major version. See [Update](#) for help in replacing existing components.

💡 DYNAMIC VS STATIC

If you want to use values from the MQTT message in UnityActions, make sure to select members from the **dynamic** section of the action (see image for an example). In the example, if you select intensity in the dynamic section it will take the value from the MQTT subscriber. If you select float intensity in the static section, you can define a fixed value to be used, no matter what the message payload is.

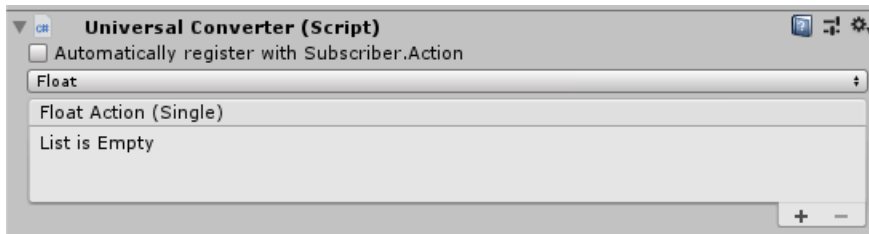


If you need to use strings from MQTT messages just use the regular Subscriber component as described in core components. However, sometimes it is desirable to interpret payloads as other datatypes or read more complex JSON payloads. For the former, SmoothMQTT brings a [UniversalConverter component](#), the latter is explained in [JSON Subscriber](#).

UniversalConverter

Type: float

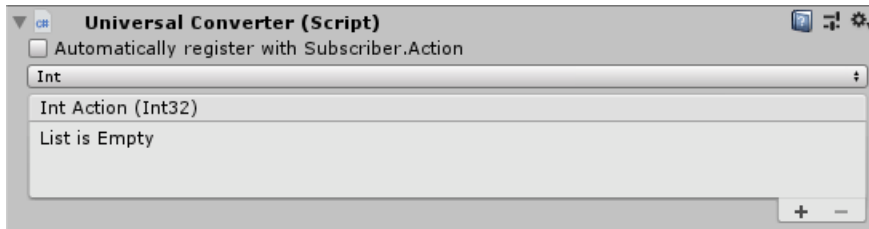
Add actions that set floats in gameobjects. For example `light.intensity`, `audioSource.volume`, etc.



Type: int

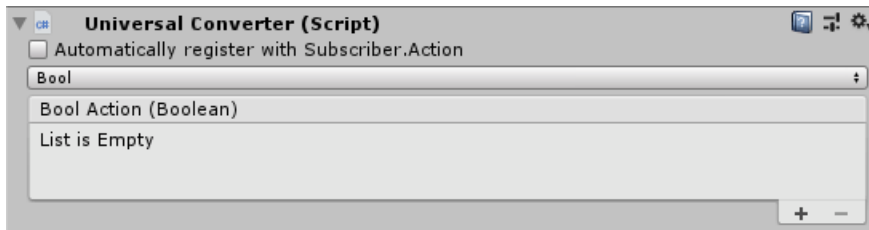
Add actions that set integers in gameobjects. Potentially used to have objects switch layers and such.

Author's remark: To be perfectly honest, I never used it, it just felt mandatory to implement it. If you find a use-case, please contact me and I will add it to the manual.



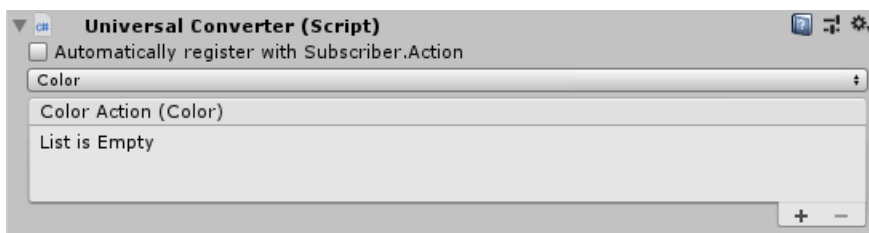
Type: bool

Add actions that set boolean values. For example activating / deactivating Components and GameObjects, toggling gravity, or pretty much anything else that shows up as checkbox in the inspector.



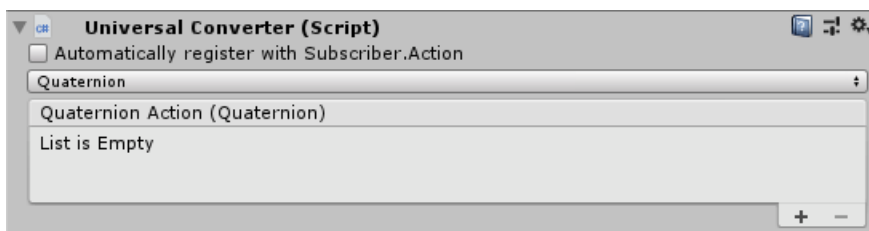
Type: Color

Add actions that react to color payloads and set color tints, light color, or material color (Caution: Changing assets like materials in Editor is permanent)



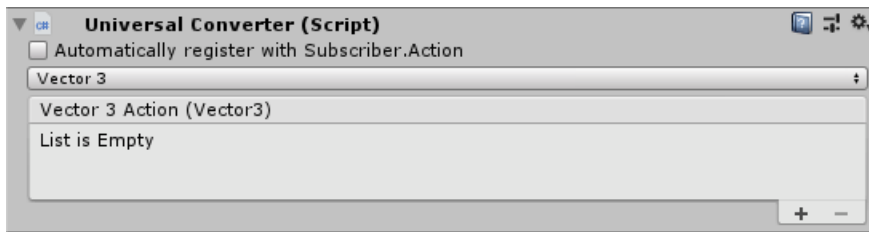
Type: Quaternion

Add actions that set a Quaternion. This is mainly used to set rotations on transforms.



Type: Vector3

Add actions that set Vector3s. This is extremely versatile and can be used to change position, scale, rigidbody velocity, navmesh agent destination, and many more things.



Conditional Subscriber

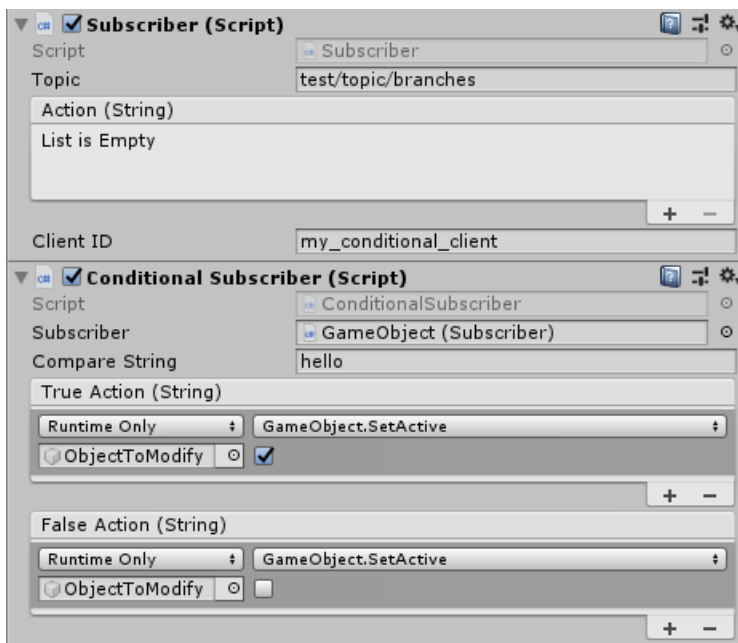
These components will provide you with an easy way for action branching. All conditional subscribers will provide you with a comparison option (more on that in the component specific part) and two UnityActions.

1. True Action (executed whenever the condition is met)
2. False Action (executed when the condition is not met) It is totally fine to use only one of the two.

Compare String (ConditionalSubscriber)

This component compares a given string with incoming MQTT messages and Invokes all the UnityActions in TrueAction if it matches and those in FalseAction if it doesn't.

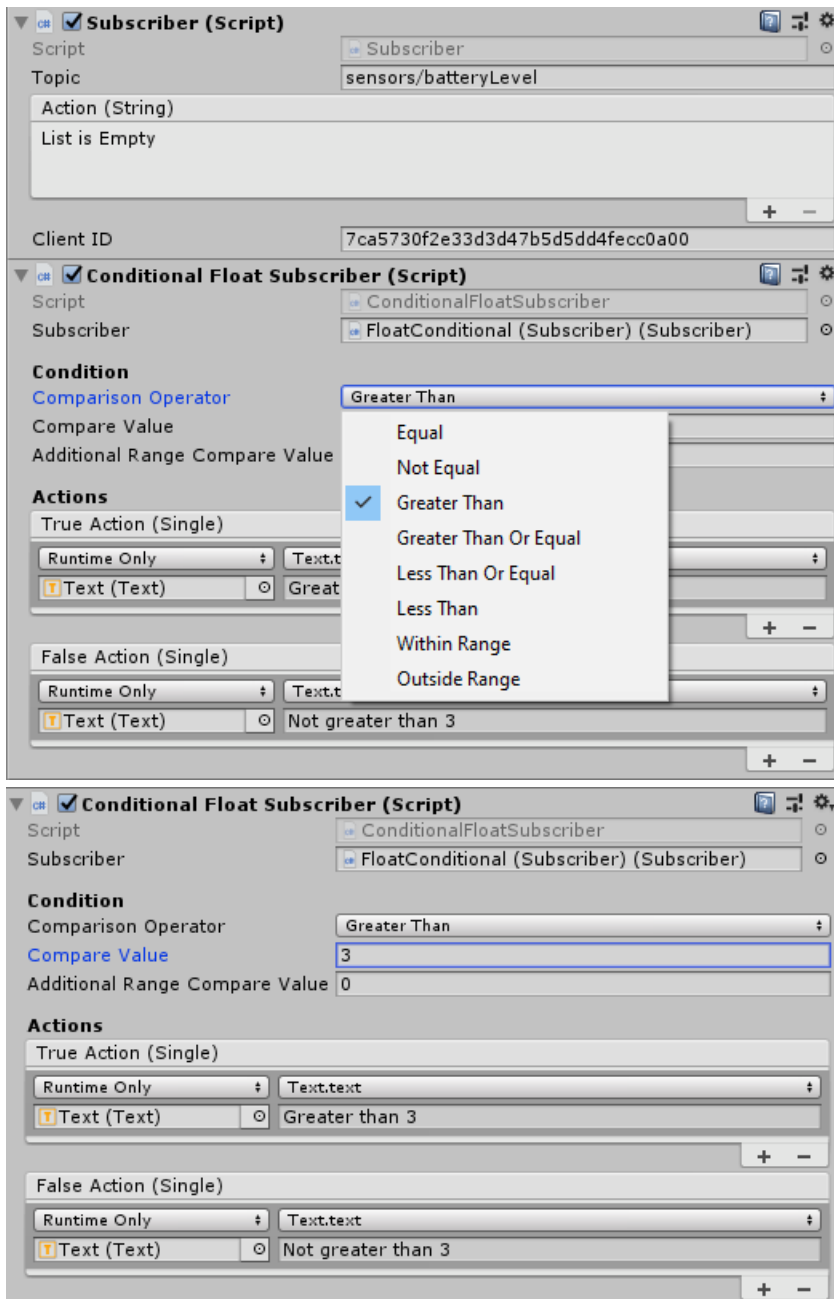
Add menu: Tools->SmoothMQTT->Receive->Compare String Subscriber



Compare float (ConditionalFloatSubscriber)

Compare incoming MQTT messages interpreted as float with a value or even a range. In the field comparison operator you can select either typical comparators like: "=", "!=", "<", "<=", ">", ">=" to compare with the value in CompareValue. WithinRange and OutsideRange allows to compare with the range defined by CompareValue and AdditionalRangeCompareValue. (Attention: a period should be used as decimal point)

Add menu: Tools->SmoothMQTT->Receive->Compare Float Subscriber



JsonSubscriber

This component allows you to access single values from JSON payloads. Similar to a regular subscriber component, you can setup the topic to listen on. In the section **Json specific parameters** in the inspector, SmoothMQTT is expecting a path to value. This is a `/`-separated string of keys (value names) and integers (index for list). But let's take a step back and look at a proper example

```
{
  "person": {
    "name": "Simon",
    "age": 47,
    "isAgeTrue": "False",
    "unityAssets": [
      "SmoothMQTT"
    ]
  }
}
```

In this JSON payload you can address the keys (value names) by their names. For example a PathToValue of `person/age` yields `47`, while `person/name` yields `Simon`, etc. If you want to access list elements (identified by square brackets `[]` in JSON) you can just use the integer index (starting at 0 for the first element). So `person/unityAssets/0` would yield `SmoothMQTT`

⚠ PAYLOAD COMPLEXITY

When working with lists, the complexity is limited to a single list index in the path. Complex nesting is not supported right away, because the interpretation works on regular expressions, to avoid third party dependencies. If you decide to install Newtonsoft.Json, though, you can click the

"Look for Newtonsoft Json" button in the inspector after installation. After a few moments it should detect the package and support more complex JSON payloads.

Like the regular Subscriber component, this component also Invokes the regular action with the complete JSON payload. If you want to continue working with the filtered value, add your listeners to "Do On Message" instead.

Finally the JsonSubscriber allows you to poll for values right away (as most off-the-shelf IoT devices have that kind of mechanism). Put in the topic and payload you have to send for polling and set the interval in seconds. If you don't want to use polling, just leave the topic empty. A more detailed description on how to use it to read temperatures from Tasmota devices will follow shortly in the Appendix of the online version of this manual:
<https://smoothmqtt.schliesky.com>

Check out this screenshot for a quick example on how to extract the age of the above example JSON payload and use it to set the font size of a text field.

If you're interested in controlling sensors without writing code, checkout [Appendix C - Tasmota](#)

Updating old components

While great care is taken that the update process is smooth (as in SmoothMQTT), usually not all edge-cases can be considered at programming time. So please make a backup of your project before you proceed.



Make a backup of your project, before you proceed!

As soon as your backup is done, click Tools->SmoothMQTT->Update Converter Components. This will replace all deprecated converter components in the currently active scene. If a gameobject happens to also be linked to a prefab, the prefab will be updated, as well. Please double check the action listeners after update.

Security Certificates (SSL/TLS)

With version 1.1.0 of SmoothMQTT support for TLS is fully implemented. And it is implemented in the typical No-Code fashion SmoothMQTT is known for. You will need to have certificates in .pfx and .crt format. If you have other formats, check the appendix for a very short step by step guide on how to convert them with openssl.

Fully implemented is unfortunately limited by the availability of functions in Mono C#. Therefore it is paramount, that you understand, there is no validation of the certificates. Certificates are loaded and used, so your data is not transmitted as plaintext, however, since there is no validation code in place, it is not considered secure from a cybersecurity point of view. Some programming is involved in getting this functionality added, which is out of scope of this asset.



TIP

A set of example certificates is included in the package, all pfx files have the password **1234**.



DANGER

DO NOT(!) use these certificates in your final project. These are for educational purposes only and are by no means secure!!!

That said, the creation process of certificates and securing it properly is expert knowledge. So if you need to have a secure connection that carries a high risk for damage or casualties when compromised, please consult an IT specialist for MQTT and certificate management. Feel free to reach out to [Simon Schliesky Softwarelösungen](#) as one option.

Minimal Requirements

To successfully encrypt your mqtt connection you need to have at least 2 certificates.

1. A client certificate containing a public and private part (.pfx format)
2. The root Certificate Authority (CA) certificate that signed the broker's certificate (.crt or .pem format)

If you get these from your broker provider you are good to go. If you get separate files for the client certificate's private and public part, check the appendix for instructions on conversion. For creating self-signed certificates check the appendix.

If you want to run the internal SmoothMQTT broker you also need: 3. A server/broker certificate containing a public and private part (.pfx format)

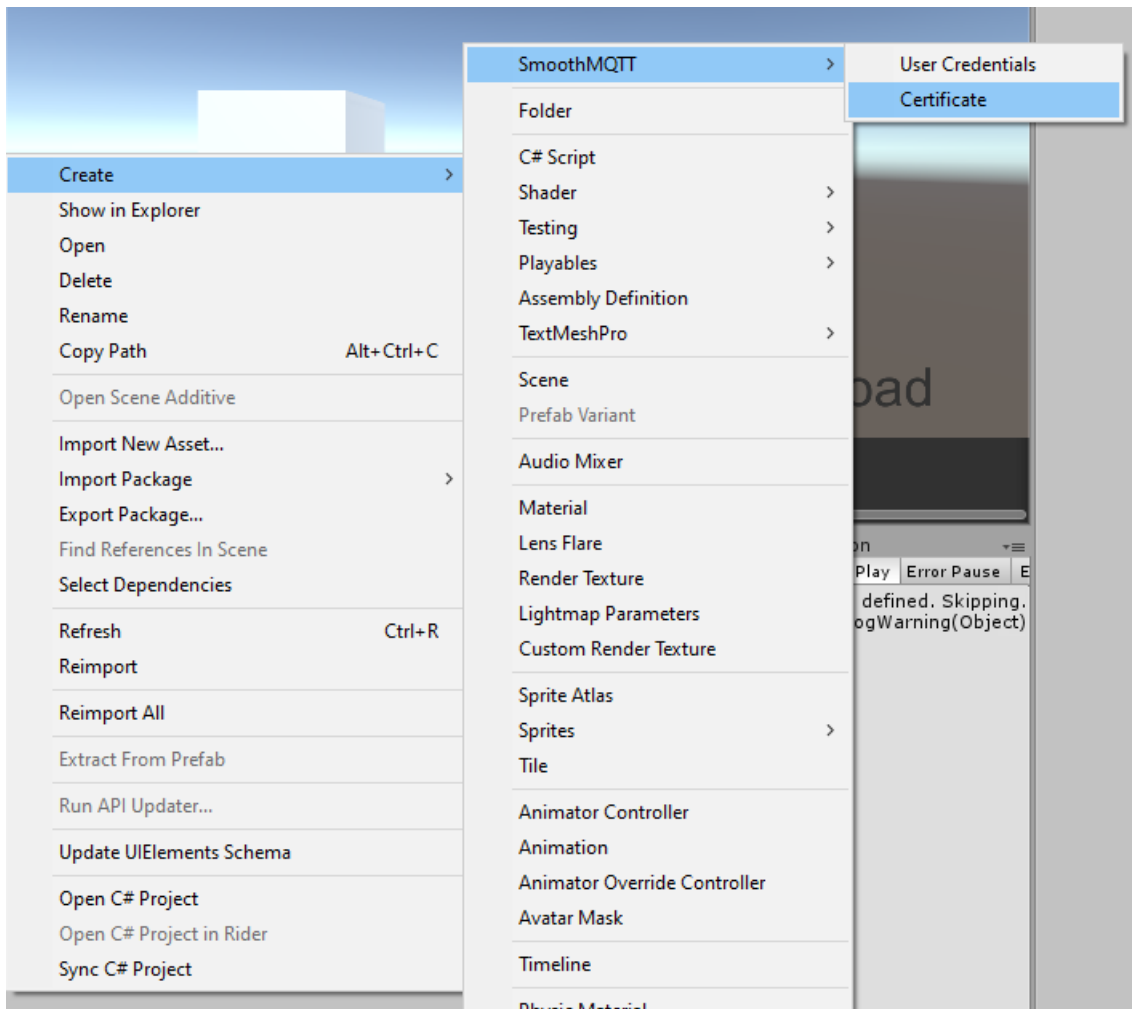
Ultimately you need to place the certificates in StreamingAssets/certificates, but worry not, there is a single click solution for you in the asset.

Example scene 101 shows the use of certificates. Before running it, the certificates need to be linked in the assets and copied to StreamingAssets though.

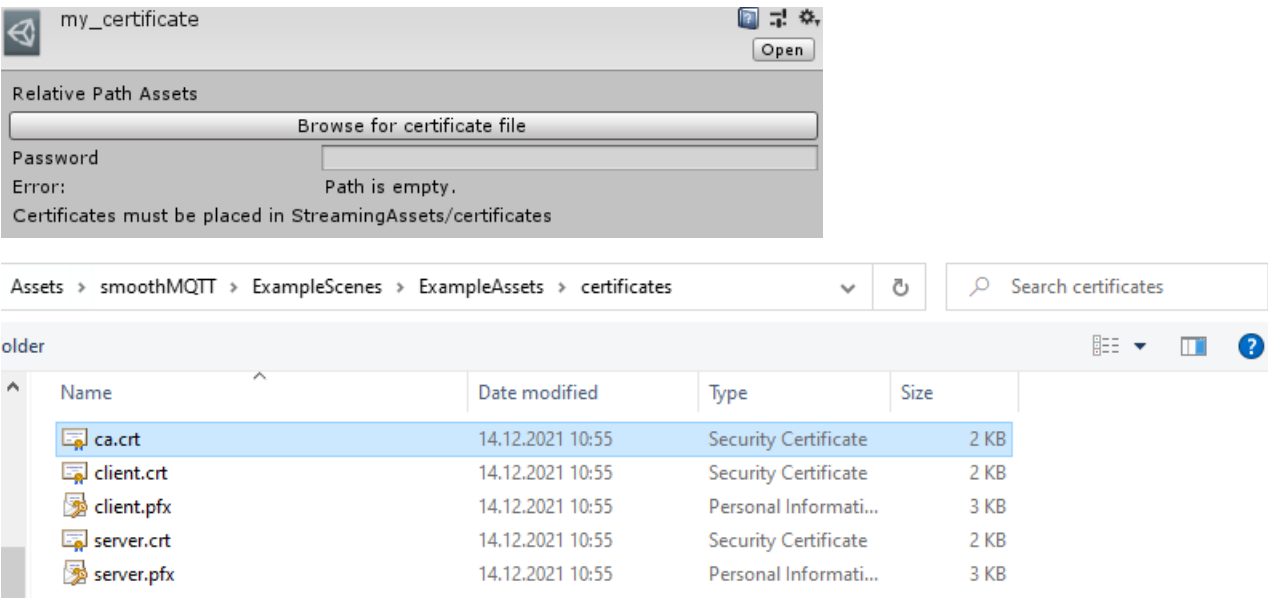
Certificate Assets

To link up certificates to smoothMQTT you will create scriptable certificates by right-clicking in your project view and selecting Create->SmoothMQTT->Certificate For each certificate you will need one. The suggested naming scheme is:

- CA certificate
- client certificate
- [broker certificate]



When you select one of these asset files, the inspector window will show a few configuration options. First and foremost, you need to set the path to your .crt, .pem, or .pfx file. Click the browse button and select a certificate file using your system's explorer.



After selecting a certificate with only a public part (e.g. ca.crt) the certificate is shown right away and can be copied to the correct folder by clicking "Copy File to Streaming Assets now."

If you select a pfx file, that includes a (by default) password protected private key, it looks like that

The certificate is not shown, because it can't be decoded without a password. As soon as you enter the correct password, it should show the certificate as seen above. Unless already done, copy the certificate file to StreamingAssets by clicking the button.

CAUTION

If there is already a certificate in StreamingAssets/certificates with the same file name and you want to replace it, you need to delete it manually first.

MQTT Settings Component

In order to use the prepared certificates during connections find your settings component (usually on the MqttManagerWith(out)Broker Gameobject) and link the certificate assets to the matching variables. CA certificate and client certificate are mandatory to use SSL. A server certificate is only necessary, if you plan on using the internal broker. Make sure to check `useSSL` and change the port according to your broker's port (default: 8883).

For the time being, SmoothMQTT can only use a single client certificate for all scripts accessing MQTT either as publisher or subscriber*.

That is pretty much it. And from here you can connect your devices with secure communication. Teaching the fundamentals of certificates is out of scope of this manual, however you may find one or two neat tricks in the Appendix A



If you need certificates per client and happen to do so in a commercial project, please [contact me](#) for a paid offer. This way you will get a good deal and everyone nice functionality :)

FAQ / Troubleshooting

Connection issues

MQTT won't connect to external broker

Usually this problem arises from an improperly configured network.

- Make sure, the external broker is available from the network you're running a smoothMQTT app in.
- Your smoothMQTT app needs to have one MQTTManager prefab instance in the scene.
- Confirm the `IP address` and `Port` on the settings component match the address and port of your broker.
- Check if the broker's firewall accepts incoming connections on the MQTT port(s)
- If it still doesn't work, restart the broker and restart the unity app.
- NOTE: If you have a very slow network connection, you might want to increase connection timeout (on settings component) to above 3, however if this is necessary your MQTT experience will be far from realtime.

I can connect but neither send nor receive messages

Check if the broker has user management and restrictions in place. Use proper username and password in settings component.

I can connect and either send or receive messages, but not both

There are user restrictions in place in the broker configuration. Make sure the client user has access to all topics to be used.

Leading Slash in Topics

This is a syntactical problem typically encountered in MQTT. The topic `/test/topic` and `test/topic` are not the same. While both are valid, some other client not from your unity app (for example a sensor or a python script) might use URL-notation for MQTT where the second slash might be autoremoved, i.e. `mqtt://user:password@hostname//test/topic` does write to `test/topic` instead of `/test/topic`. This is very hard to find and debug, so my suggestion is to **never use a leading slash in your topics**.

Tools Menu does not do anything

Check your console for any warnings. If it says "No Gameobject selected" you need to select a gameobject in your scene or hierarchy before trying to add the components.

Android Builds

1. Make sure that the Android platform is activated in Assets/smoothMQTT/Scripts/SmoothMQTT/SmoothMQTT.asmdef. By default, it is deactivated, because it is not an officially supported platform. (Alternatively you can click on Tools/SmoothMQTT/Activate Android. It will try to switch to Android and change the asmdef file for you.)
2. Confirm that the API Level is set to 29 or higher. Although it may work with lower levels, there are no guarantees.
3. Install a debug build (while building check "development build" and "script debugging") on your phone and check the logcat output after starting the app. To do this, you will need to enable USB debugging and use a tool such as Android Studio to read the logcat logs. Filter the logs by your app name as defined in the Player Settings.

Menus Available in SmoothMQTT

Gameobject

Entry	Description	When to Use
Gameobject		
↳ SmoothMQTT	Instantiates the equally-named prefab into your current scene	Run everything MQTT from within Unity
↳ MqttManager With Broker		
Gameobject		
↳ SmoothMQTT	Instantiates the equally-named prefab into your current scene	Use external Broker, e.g. smarthome, mosquitto or cloud provider
↳ MqttManager Without Broker		



TIP

If you are unsure about using an external broker, we suggest you use the first option. If you later on decide to use an external broker, you can simply deactivate the broker component.

Tools

Entry	Description	When to Use
Tools		
↳ SmoothMQTT	path to the following menu entries (to declutter the table) ...	
↳ Receive		
↳ String payload	Adds a Subscriber component with random client ID to the selected GameObject	- Setting Text or TextMeshPro UI elements - Setting animation triggers by name
↳ Float payload	Adds a Subscriber component with random client ID to the selected GameObject and adds a <code>float</code> UniversalConverter to interpret the payload as decimal number and pass it onto components using float values	- light intensity or range - gravity scale on rigidbodies - sphere collider radius, nav mesh agent speed ...
↳ Int payload	Adds a Subscriber component with random client ID to the selected GameObject and adds a <code>int</code> UniversalConverter to interpret the payload as whole number and pass it onto components using int values	- mainly for custom scripts - gameObject layer
↳ Bool payload	Adds a Subscriber component with random client ID to the selected GameObject and adds a <code>boolean</code> UniversalConverter to interpret the payload as truth value and pass it onto components using bool values	- enable/disable gameobjects and components - pretty much anything that shows as checkbox in inspector
↳ Color payload	Adds a Subscriber component with random client ID to the selected GameObject and adds a <code>Color</code> UniversalConverter to interpret the payload as a color value and pass it onto components using colors	- camera background color - sprite/UI tint - light color
↳ Vector3 payload	Adds a Subscriber component with random client ID to the selected GameObject and adds a <code>Vector3</code> UniversalConverter to interpret the payload as a vector and pass it onto components using vectors	- position, rotation, scale, LookAt - nav mesh agent destination - collider size
↳ Quaternion payload	Adds a Subscriber component with random client ID to the selected GameObject and adds a <code>Quaternion</code> UniversalConverter to interpret the payload as a rotation and pass it onto components using quaternions	- rotation - rotation on directional light for daytime
-----	-----	-----
Tools		
↳ SmoothMQTT	path to the following menu entries (to declutter the table) ...	
↳ Send		
↳ String payload	Adds a Publish Value component to the selected GameObject that can be called from any UnityEvent to send <code>string</code> , <code>int</code> , <code>float</code> , or <code>bool</code> values.	Use buttons or unityevents to send arbitrary values to e.g. smart light bulbs or smart sockets.
↳ Color payload	Adds a Publish Color component to the selected GameObject that can be called from any UnityEvent to send a hexadecimal representation of a color.	Send color value to e.g. LEDs
↳ Vector3 payload	Adds a Publish Vector3 component to the selected GameObject that can be called from any UnityEvent to send the string representation of a vector.	Send vector3 value (mainly used to communicate with other SmoothMQTT components)
↳ Quaternion payload	Adds a Publish Quaternion component to the selected GameObject that can be called from any UnityEvent to send the string representation of a quaternion.	Send static rotation or send rotation of a transform (mainly used to communicate with other SmoothMQTT components)
↳ TMPInputField	Adds a Publish From TMP Input Field component to the selected GameObject that can be called from any UnityEvent to send the string representation of a quaternion.	Send content of a TextMeshPro Input Field. This component's <code>OnSendInputField</code> function is supposed to be called from the <code>On End Edit</code> event on a TMPInputField
↳ React to UnityEvents	Adds a Send On Event component to the selected GameObject. This component sends a static payload to the defined topic upon gameObject event triggers	This is great for click or drag events on objects in the scene. Use-cases are: click a virtual button to switch on LEDs or motors. Hover over an object and show the tooltip on a 16x2 liquid crystal display.
↳ React to Trigger or Collision	Adds a Publish Trigger Collision component to the selected GameObject. This component can send the description of a collision to a dynamically configurable topic	Switch on/off fans or heaters when player enters triggerzone (windy mountain top, scorchnign desert). Also great for detecting object collisions and switching on alarm lights. We sincerely hope no horror game will ever use SmoothMQTT for flickering the room lights.

Assets

Entry	Description	When to Use
Create ↳ SmoothMQTT certificate (e.g. path, password) ↳ Certificate	Creates a scriptable asset to hold information to a SSL	When you want to encrypt communication between Unity and MQTT broker through mqttts://
Create ↳ SmoothMQTT ↳ User Credentials	This very simple object stores a user and password to be used by the local SmoothMQTT broker	When you require username and passwords from inbound connecting clients, create one of these assets per user/password pair

A - Creating and Converting Certificates

All commands in this section use [openssl](#) which is readily available for Windows, Linux and OS X.



TIP

On windows you might need to run openssl.exe instead of openssl (in the shell commands below)



INFO

shell, *commandline* and *terminal* are used interchangeably in this section ...

If you came to this section, it is assumed, that you are in general aware of how to run commands on the commandline of your computer.

Creating your own CA certificate

```
# In shell run these lines one after the other (because some need interaction)
openssl genrsa -des3 -out ca.key 2048
openssl req -new -x509 -days 1826 -key ca.key -out ca.crt
```

These commands create a ca key and certificate. The certificate is supposed to be shared with all devices. Keep the key secure, it is used to sign other certificates. Hence the name CA (Certificate Authority)

Creating a computer certificate and sign it with CA

This time around you create a key and a certificate signing request which is then signed by the CA to create a certificate.

```
openssl genrsa -des3 -out my_client.key 2048
openssl req -new -out my_client.csr -key my_client.key
```

Send/upload this .csr file to the CA for signing. Unless you ARE the CA, then run

```
openssl x509 -req -in my_client.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out my_client.crt -days 360
```

Converting a .crt/.pem and a .key file to a .pfx file

```
openssl pkcs12 -export -out my_client.pfx -inkey my_client.key -in my_client.crt
```



TIP

The export password you set here is the password you will need to set in the SmoothMQTT certificate asset (see [Certificate Assets](#)).

B - MQTT in the Cloud & AWS* IoT Core

Many users asked whether SmoothMQTT works with cloud services. Now that SSL is implemented it should work with any cloud service that provides MQTT over Sockets. Here you find a very basic and not super secure approach on setting smoothMQTT up to work with AWS IoT Core. If you need proper security, please always hire a security specialist. These exact instructions are meant for proof of concept only.

1. Login to IoT Core
2. Go to Secure-Policies and select "create Policy"
 - i. As name choose something like "smoothMQTT"
 - ii. As action type "iot:*"
 - iii. As resource type "*"*
 - iv. Check "Allow"
 - v. Click "Create"
3. In the menu select Manage->Things and Create a Single Thing
 - i. Give it a name e.g. "smoothMQTT-client"
 - ii. Click "Next"
 - iii. Check "Auto-generate a new certificate"
 - iv. In the attach policies screen select the policy named in 2
 - v. Download the public and private key for your client
 - vi. Download the RSA2048 Amazon Root CA public certificate
4. Use openssl to convert the public and private key to a pfx file (see Appendix A)
5. In Unity create two certificate assets (see chapter 5.2)
 - i. Name one "CA cert" (or something similar) and select the amazon root CA file. Copy it to StreamingAssets using the inspector button
 - ii. Name the other one "client cert" and link the pfx file created in step 4.
6. Create a scene with SmoothMQTT's MqttManagerWithoutBroker
 - i. Link up certificate assets to the settings
 - ii. As host insert the Device data endpoint address. It looks like "[something.something.amazonaws.com](#)" (You'll find the correct address in the Settings on IoT Core)
 - iii. Set port to 8883
 - iv. Set useSSL to true
7. Create a publisher or subscriber with a topic (e.g. smoothmqtt/test) and [optionally] a payload.
8. On IoT Core open the MQTT test client and subscribe to "smoothmqtt/test" or even "#" and see whether your unity published messages arrive. If you used a subscriber in unity, select "Publish to a topic" on IoT Core and send a message.

Security considerations: If you want to build a secure business solution, you will need to setup proper policies for IoT core, so certain clients can only interact with certain topics and actions. However, a detailed description is out of scope of this manual.

! INFO

*AWS (Amazon Web Services) is a registered trademark in the USA and other countries.

C - Code-Free MQTT-Devices Using Tasmota

Ever since creating SmoothMQTT, I was searching for a good code-free possibility to install and setup IoT devices. Then I found [Tasmota](#) and it is amazing.

For those who may have limited time and/or attention span, please find the [TL;DR](#) for a condensed summary of the information at the bottom of this page.

❗ DISCLAIMER

Tasmota does not endorse SmoothMQTT or has in any way compensated me for including it here. Frankly, they probably don't even know about SmoothMQTT. I just love this Framework and believe it contributes to the same vision of lowering the entry barrier for working with IoT devices, just like SmoothMQTT does.

What is Tasmota

If you're interested in building and managing your own smart home or tinkering with smart devices for your games without coding Tasmota has you covered. Tasmota is an open-source firmware that can be used to control various types of smart home devices such as lights, outlets, and sensors. It's a popular choice among smart home enthusiasts and DIYers because it is easy to use and highly customizable. With Tasmota, you can configure your smart home devices to behave in specific ways based on certain triggers or conditions. For example, you can set up a rule to turn on the lights in your living room when you open the front door, or you can create a timer to turn off your coffee maker after it's been on for a certain amount of time. The possibilities are endless, and Tasmota makes it easy to get started with your own smart home projects.

This manual will focus on the Tasmota basics and show you how to connect and use Tasmota smart devices together with SmoothMQTT. Therefore, after reading through these instructions, you should be able to use data from a simple sensor like for example BMP180 (temperature and humidity) or similar sensors, as well control for example Tasmota sockets (switch on or off, depending on your Unity scene).

Setting up Tasmota:

Chrome users

If you happen to use the Chrome browser, there is a very convenient way of flashing your ESP32 with Tasmota firmware.

1. Connect it with USB to your computer
2. Go to <https://tasmota.github.io/install/>
3. Follow the instructions on screen for flashing Tasmota

Other browsers

Here is a step-by-step guide to setting up Tasmota on a blank ESP32 devkit using the commandline:

1. Download the [Tasmota firmware file](#) from the Tasmota website. Make sure to download the correct version for your ESP32. If you do not have specific requirements, go for the tasmota32.bin file.
2. Install esptool with `python3 -m pip install esptool`. Note: you will need Python3 installed on your system. The official documentation can be found [here](#).
3. Connect the ESP32 devkit to your computer using a micro-USB cable.
4. Find out the port of your device. On windows you can check the device manager, the port usually has the format COMX (where X is a number). On macOS the port is usually called /dev/cu.usbserial.X (where X is a number), find it with `ls /dev/ | grep "usb"` on the Terminal.
5. Open a Terminal in your Downloads folder
6. [optional] Run `python3 -m esptool -p [name of your port] read_flash 0x00000 0x100000 fwbackup.bin` to create a backup of the current firmware
7. Run `python3 -m esptool -p [name of your port] erase_flash` to delete the ESP memory
8. Run `python3 -m esptool -p [name of your port] write_flash 0x0 tasmota32.bin` to write the Tasmota firmware onto your ESP32. Note: For other boards it might be different, if in doubt check the Tasmota documentation.

At this point, your ESP32 is now running Tasmota and is ready to be configured and used to control your smart home devices. In the next section, we'll cover the steps for setting up and accessing the Tasmota web interface.

Configuring Tasmota for the first time:

Now that you've set up Tasmota on your device, you'll probably want to customize the settings to suit your needs. In this section, we'll go over the various options available in the Tasmota web interface and how to use them to configure your Tasmota device. The official Tasmota manual on how to configure the device can be found [here](#) under the tab Web-UI.

Configuring WiFi

1. Connect your ESP to a power source.
2. On the first start, it will create a wireless access point. The name will be something like tasmota_XXXXXXX-YYYY
3. Use some device (in this example a phone) to connect to this WiFi access point.
4. Wait for the captive portal to open or if it doesn't go to <http://192.168.4.1> in your phone browser.
5. Here you can enter (or click if listed) your WiFi network name (SSID) and password to connect your Tasmota device to your local network.
6. Once it is connected, it will show you the IP address of the ESP in the browser. It depends on your network, but might look like 10.0.0.123. Write that down.
7. Connect your phone back to the normal WiFi.

Now your ESP is successfully connected to your WiFi. Continue with configuring the MQTT part.

Configuring MQTT

1. Go to `http://<ip>`, e.g. <http://10.0.0.123>
2. Enter the Configuration -> MQTT menu (see Screenshot)
3. Set Host to the MQTT broker address
4. Set the client ID in "client"
5. Set Username and password if applicable.
6. Set the topic to something that helps you remember it (e.g. sensors/livingroom/temp_hum)
7. Leave full_topic as is, an explanation follows below
8. Click "Save" to apply your changes.

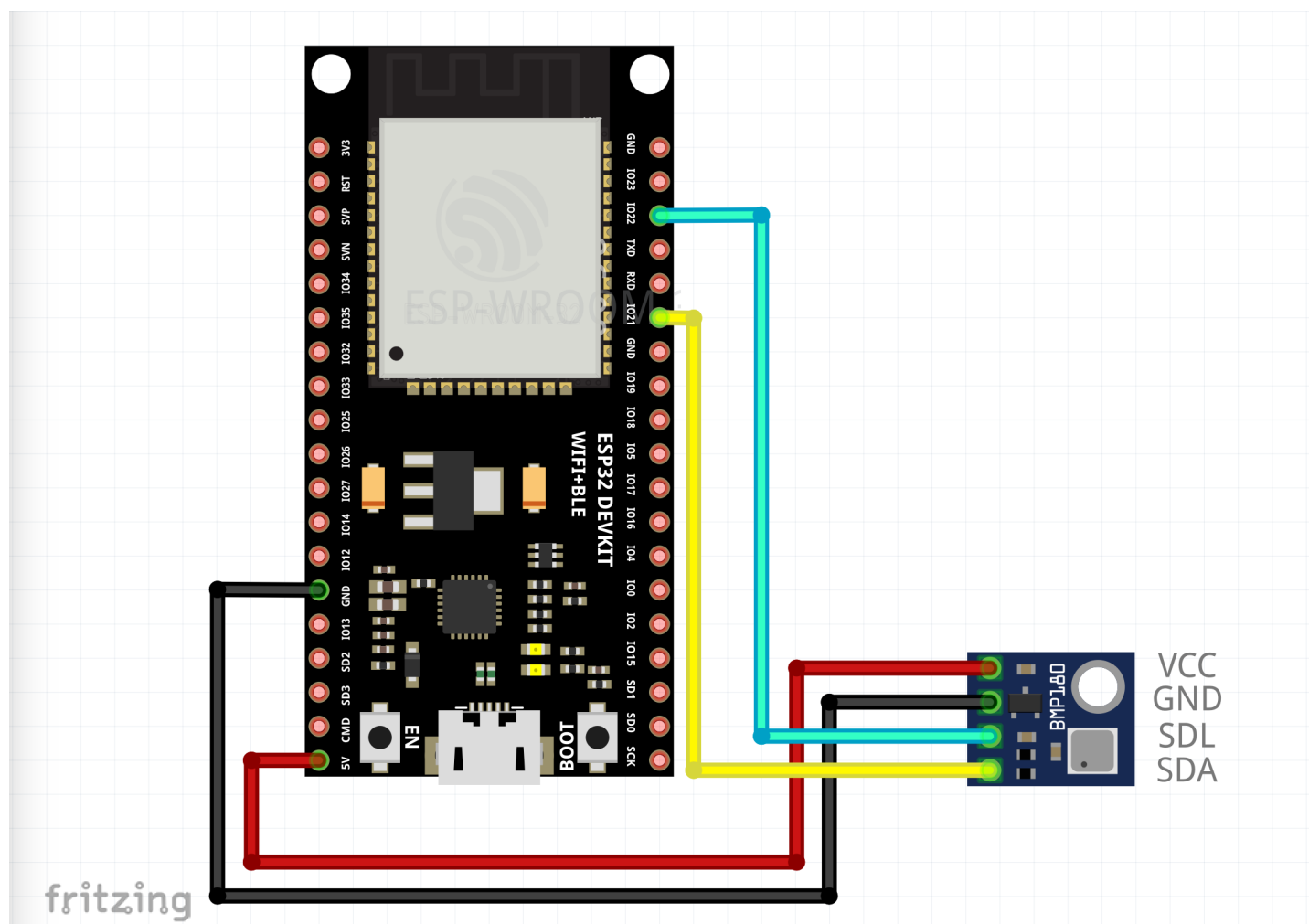
The screenshot shows the Tasmota web interface for an ESP32-DevKit. The title is "Tasmota". Below it is a section titled "MQTT-Einstellungen". This section contains several input fields: "Host ()" (empty), "Port (1883)" (1883), "client (DVES_7BF344)" (DVES_%06X), "Benutzer (DVES_USER)" (DVES_USER), "Passwort" (masked with dots), "topic = %topic% (tasmota_7BF344)" (tasmota_%06X), and "full topic (%prefix%/ %topic%/)" (%prefix%/ %topic%/). Below these fields is a green button labeled "Speichern". At the bottom of the interface is a blue button labeled "Einstellungen". The footer text reads "Tasmota 10.1.0 von Theo Arends".

That's it! You've now configured Tasmota for the first time and are ready to start using it to control your smart home devices. In the next section, we'll discuss how to use Tasmota together with SmoothMQTT to control your devices.

Examples

Measuring temperature and humidity with BMP180

To use a BMP180 sensor with Tasmota, you'll need to connect it to the ESP32. Follow the wiring diagram provided with the BMP180 to connect the sensor to the ESP32 using the GND, VCC, SDA, and SCL pins. Please make sure the ESP is not powered, before you start wiring anything. Typically SDA and SCL connect to pin 21 and 22 respectively.



🔥 WIRING

Double check the pins, as the pinout of your BMP180 board or your ESP32 may differ. Wiring the wrong pins can destroy your boards.

Once the BMP180 is connected to the ESP32, power it again and go to the web interface. You'll need to configure Tasmota to recognize the sensor. Therefore, in the Tasmota web interface, go to the "Configuration" tab and select "Configure Other". Scroll down to the "GPIO21" and "GPIO22" options and enter the "I2C SDA" and "I2C SCL" respectively. Then, click "Save" to apply your changes.

Now that the BMP180 is connected and configured, you can use Tasmota to access the temperature and humidity readings from the sensor. You can display the readings on the Tasmota web interface, or you can use MQTT to send the readings to a different device or service. For example, you can set up a rule to send an alert to your phone if the temperature exceeds a certain threshold, or you can create a timer to send the humidity readings to your game on a regular time interval. However, we'll not discuss Rules in-depth.

Below the other example is a guide on how to use data from this configured tasmota device

Controlling a smart socket

If you want to switch power on and off for example on a fan, you can use pre-flashed smart sockets which use tasmota. In Europe [Nous A1T](#) is a good choice. I'm not affiliated with them and I don't know if they ship worldwide though.

Configure the smart socket just like the ESP32 above by connecting to it and setting up WiFi and MQTT. Afterwards, you can use the Tasmota

WebUI to toggle the socket.

Controlling tasmota devices with SmoothMQTT

First, make sure you have the SmoothMQTT plugin installed in your Unity project. You can find it on the Unity Asset Store or download it from the SmoothMQTT website.

Once you have SmoothMQTT installed, you'll need to add the JsonSubscriber component to a gameobject in your scene. You can do this by selecting the gameobject, then going to the menu "Tools/SmoothMQTT/Receive/Json Payload". This will add the JsonSubscriber component to the gameobject, which you can use to receive messages from the Tasmota device.

To configure the JsonSubscriber component, you'll need to enter the correct topic that the Tasmota device is publishing to. This will typically be in the form of "stat/[topic name]/[reading]", where [topic name] is the name of your Tasmota topic defined in the config and [reading] is the type of message you want to receive. For example, you might use a topic like "stat/livingroom/socket1/power" to receive power status messages from a Tasmota-powered socket in the living room. To access the aforementioned temperature and humidity sensor, you would subscribe to "stat/livingroom/temp_hum/status10"

Why status 10?

Tasmota allows for different commands to control what kind of status the ESP will return upon being polled for reading. Number 10 simply is the format, where the ESP returns a time stamp and all sensor readings available (see tasmota Docs for reference). To set up a polling topic, simply enter the topic and payload that you want to receive updates for in the "Poll Topic" and "Poll Payload" fields and enter the time interval in the "Poll Interval" field. To get this Status10 reply mentioned above, the poll topic is "cmnd/livingroom/temp_hum/status" and the payload is "10". So giving a tasmota device a command is prefixed with "cmnd/" and reading the returned values is prefixed with "stat/".

Once you've set up the JsonSubscriber component with the correct topic and optional polling settings, you need to tell the component how to find data in the json that is returned. Check the manual entry [JsonSubscriber](#) for instructions.

To control the smart socket from example 2, just send a regular MQTT message to topic "cmnd/livingroom/socket1/power" with the payload "on", "off", or "toggle"

Overall, using SmoothMQTT in Unity to connect to a Tasmota device is a simple and straightforward process. With just a few clicks and a little bit of configuration, you can easily integrate Tasmota into your Unity projects and use it to control and monitor your smart home devices. Despite the fact tasmota rules will bring you a lot of possibilities to react to the environment, it is perfectly possible to create engaging experiences without writing code when using SmoothMQTT.

Imagine for example a VR experience, where the player enters a triggerzone (mountain summit for example) and in their room the fan socket switches on and provides wind to add to player immersion. Imagine infrared bulbs to support the illusion of entering a desert. Imagine a heart rate sensor's readings open certain doors in a dungeon crawler. Build a custom hardware for your trade fair booth. The possibilities are endless.

Conclusion

With this appendix you will be able to integrate most I2C or typical arduino sensors into your SmoothMQTT app. You will be able to control devices or monitor them. You can use the sensor readings to control the flow of your game, even switch scenes when certain events happen. And last but not least, you can build custom hardware that is easily integrated into your experience, as all the MQTT components provide UnityEvents to react to MQTT messages.

TL;DR

For custom sensors:

1. Connect your Computer via USB with an ESP32/8266 devkit
2. Go to <https://tasmota.github.io/install/>
3. Follow the instructions to install the default version
4. Connect the sensor to GPIO pins and remember which ones

Checkout <https://tasmota.github.io/docs/Getting-Started/> for more information

For shelf-bought Tasmota devices or the custom ESP after the previous steps:

1. Power the device.
2. Look for a new wifi accesspoint (usually tasmota-XXXXXX) and connect
3. If it does not open a captive portal automatically, open a browser at <http://192.168.4.1>
4. On the page enter your WiFi credentials to allow the ESP to connect to your internal network.
5. After connection you're either redirected or presented with an IP address (in your local network) open this address in your browser.
6. Go to configuration->MQTT and set your host (IP of the MQTT broker), user/password if applicable, and a topic

::: Note The topic set here is really usually the device name, because MQTT topics are auto-generated from these. So the topic `office_temperature1` would lead to e.g. `stat/office_temperature1/power` or `office/socket/fan` (yes slashes are valid here) would yield e.g.

`cmnd/office/socket/fan/power` ::: Checkout <https://tasmota.github.io/docs/MQTT/> for more information on configuring MQTT with tasmota.

7. Once saved, your tasmota-device will send telemetry data in certain intervals (configuration is somewhere in the WebUI) and can also be prompted for data.
8. Sending a message to `cmnd/device-topic/command` tells the device to do something
9. `stat/device-topic/Something` will yield data
10. The most important command for sensors is `cmnd/device-topic/status` with payload `10` this will tell the Tasmota device to publish its data to `stat/device-topic/status10`. Data will be JSON formatted.
11. The most important command for smart power outlets is `cmnd/device-topic/power` with a payload of either `on`, `off`, or `toggle`

For everything available commands, check <https://tasmota.github.io/docs/Commands/#commands-list>



PERSONAL SUPPORT FOR TASMOTA IS LIMITED

Please be aware that these instructions are provided free of charge to connect hardware to your SmoothMQTT app without having to code in C++ or python. This also means, however, there will be no unpaid email support for troubleshooting tasmota. Support is limited to SmoothMQTT and its associated components. For questions about Tasmota, please refer to the usual sources of help such as the [Tasmota Docs](#), [Stack Overflow](#) (make sure to use tag tasmota), or [Reddit](#).

Roadmap

- ☒ Add Conditionals ($>$, $>=$, $<$, $<=$, $!=$, $==$) for
 - ☒ float messages
 - ☐ int messages (should work fine with float interpretation, but technically not implemented yet xD)
- ☒ Evaluate reasonable uses of Conditional Colors (is in Gradient or not)
- ☐ Settings functions and UI dialog prefab to change mqtt server and credentials and stuff
 - ☐ PlayerPrefs integration for settings
- ☐ Add Formatstring option to subscriber (Not sure about the need with all the new converters)
- ☒ Create interactive manual scene based on MQTT showing possible use-cases. (not viable, use-cases too different)
- ☒ Add JSON parser for subscribing to values inside complex payloads
- ☒ Write manual for using SSL (including certificate creation a.s.o.)
- ☒ Improve Documentation formatting
- ☒ Validate Credentials when using internal Broker
- ☒ Add component for publishing messages from Trigger and Collision events
- ☒ Subscription Converters for other data types than string
- ☒ Publish Components for Vector, Quaternion, Color
- ☒ Publish Component for Text Mesh Pro InputField

Features v1.3

- mqtt:// and mqtts:// (SSL) protocols
- Unity MQTT clients
- Unity MQTT broker
- Configurable IP, port, and user credentials
- Message publishing
- Message subscription
- Primitive data types in messages (int, float, bool, string)
- Unity types in messages (Vector3, Quaternion, Color)
- Conditional subscriptions (i.e. compare message to string and do A or B)
- Interpreting values from JSON payloads
- Publishing messages from a TMP_InputField
- Publishing trigger and collision events
- Setting up user/password if Unity app is the broker
- Conditional subscribers (>, >=, <, <=, !=, ==, in interval, outside interval) for float messages
- Conditional Gradient to test if payload string is hex color within a certain gradient
- Auto Reconnect and persistency across scenes



THIS PACKAGE DOES NOT YET SUPPORT

- Websockets

Supported Platforms

⚠️ NOTE

Only those tagged as tested are officially supported

- Windows 10: tested and works
- Windows 11: untested
- OS X: tested and works
- Linux: tested and works

Not (officially) supported

- Android: Only tested with Android 10 (Api Level 29) on a Pixel 5 with external broker. So officially not supported, even though it has been seen working.
- iOS 16: untested
- WebGL: not supported (because Websockets are not supported)
- Universal Windows Platform: untested

🔥 ANDROID

For Android you need to activate the platform in the assembly definition file (Assets/smoothMQTT/Scripts/SmoothMQTT/SmoothMQTT.asmdef) or click the menu Item Tools->SmoothMQTT->Activate Android

Unity versions

- 2019.3 (LTS): tested
- 2020.3 (LTS): tested
- 2021.3 (LTS): tested
- 2022.1: untested

🏠 Need help?

Version: 1.3

Need help?

Things don't always go as planned.

And for those times, that even the [FAQ](#) don't help, there are two ways to get additional help

Online Form (recommended)

<https://forms.gle/UY5xxiB72GuGdeVj8>

Email

You may also send in your support request to p-assetstore@schliesky.com just be aware that response times are slower than with the form.

For emails please include:

1. Unity Assetstore Invoice Number
2. Unity Version
3. SmoothMQTT version
4. Target build platform(s)
5. A detailed description of what you tried, what happened and what you expected to happen. Please include as much detail as possible (e.g. inspector settings for all components used, steps to reproduce a bug (if applicable), topic names and debug/console output)
6. Relevant screenshots
7. [if available] link to an unlisted Youtube video that shows the problem

Support Scope

Please be aware that support only applies to the usage and operability of SmoothMQTT. Using SmoothMQTT together with third party software solutions, like external brokers or cloud solutions can lead to exponentially more problems than the core functionality.

However, if you find the need for support outside of this official scope, don't hesitate to send a business inquiry to p-assetstore@schliesky.com. Describe your endeavor, the budget available, and the country of origin. I will check the tax regulations and let you know, whether I can take on the project or not.